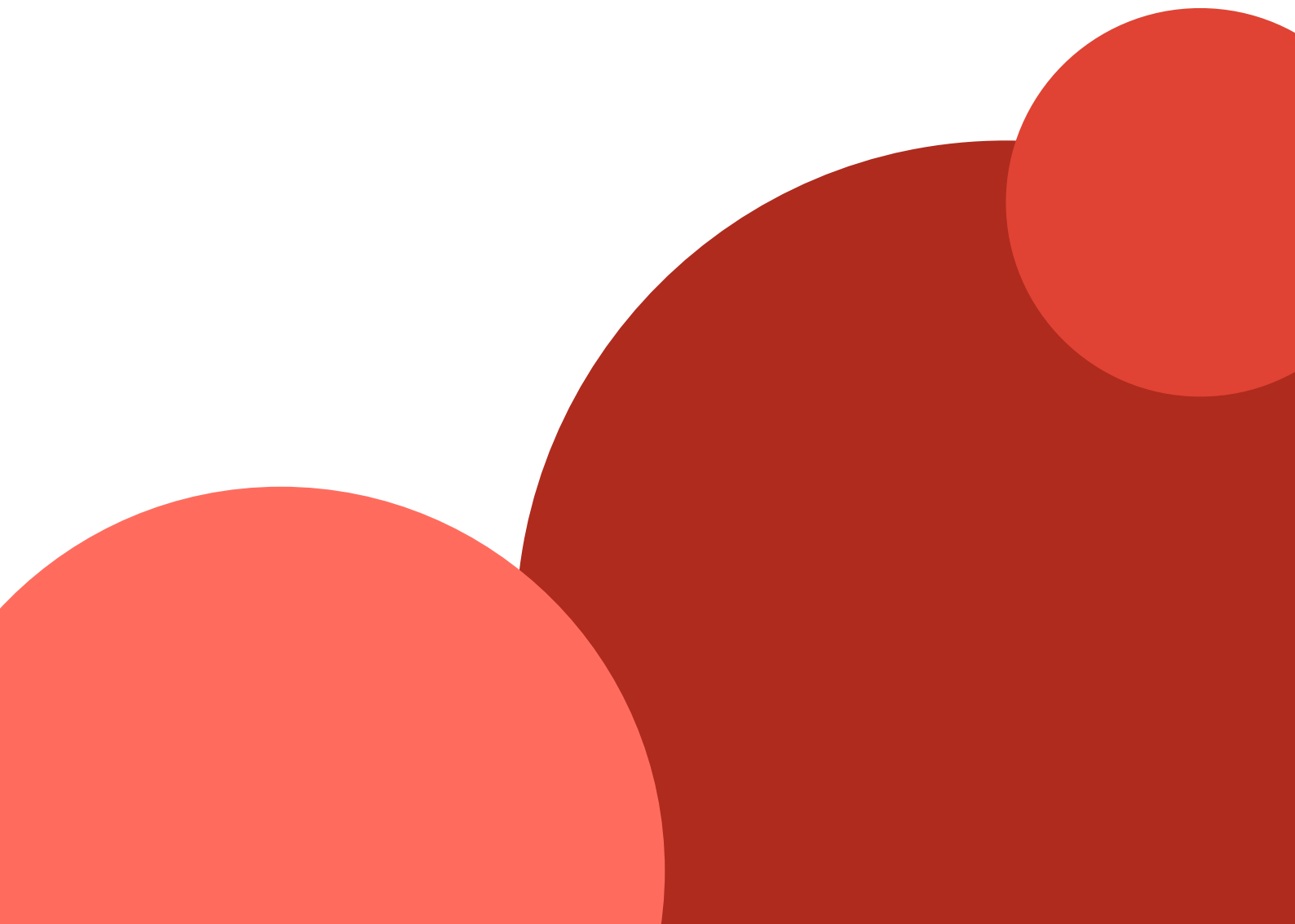


POS1-Dokumentation-TimeTravel



1 . Lastenheft

2-er Team

Efe Yasar

Furkan Yildiz

Spielidee:

Grundidee:

- Ein Junge findet im Dachboden seines Opas ein altes Tagebuch. Beim Durchblättern wird er in verschiedene Epochen teleportiert. Um aus jeder Epoche zu entkommen, muss er zeitbasierte Rätsel lösen, die mit der Kultur, Technik oder Mythologie der jeweiligen Zeit zusammenhängen.
- Jedes gelöste Rätsel schaltet eine neue Tagebuchseite frei – und damit die nächste Zeitreise.
- Logikrätsel
- Kombinationsrätsel
- Interaktive Objekte (Hebel, Symbole, versteckte Hinweise)

Story im Groben:

- Der Opa des Jungen war ein Zeitreisender, der seine Reisen in einem geheimnisvollen Tagebuch dokumentiert hat. Jede Seite enthält kryptische Symbole, Zeichnungen und Hinweise.
- Als der Junge das Tagebuch berührt, wird er in eine vergangene Epoche gezogen.
- Dort merkt er schnell:
- Der Opa hat überall Rätsel hinterlassen, die nur jemand lösen kann, der das Tagebuch besitzt. Diese Rätsel dienen als „Schlüssel“, um die Zeitreise weiterzuführen.
- Der Junge muss:
- Hinweise im Level finden
- Symbole im Tagebuch entschlüsseln
- Mechanismen aktivieren
- Zeitmanipulationen anwenden
- Logische Zusammenhänge erkennen
- Erst wenn das Rätsel der Epoche gelöst ist, öffnet sich ein Zeitportal zur nächsten Zeitperiode.
- Am Ende erfährt er, warum sein Opa diese Rätsel gebaut hat – und wie er die Zeitschleife endgültig verlassen kann.

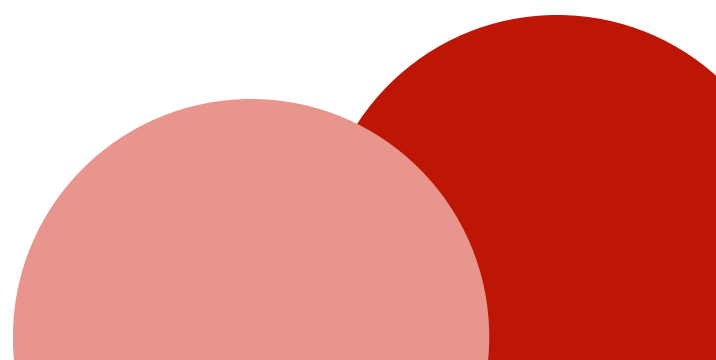
Steuerung:

- W - Vorwärts
- S - Rückwärts
- A - nach links
- D - nach rechts
- E - Interagieren
- Mausklick - Interagieren

Must-Haves:

- Spielbarer Protagonist
- Steuerung oben genauer beschrieben
- Speichern
- Laden
- Also funktionierende Serialisierung
- Level
- Fortschritt
- ordentliche und übersichtliche Ordnerstruktur
- ein Level Design
- Anklickbare Schlagwörter/Bereiche im Tagebuch, um die Zeitreise auszuführen
- 1 Zeitreise
- Animationen

Nice-To-Haves:

- Story
 - tolle Grafik und Sound
 - Flüssiges Spiel
 - dazu zählen u.a. so wenig Bugs oder Abstürze wie möglich
 - Easter Eggs/Details
 - schönes Level Design
 - tolles Gameplay
 - freigeschalteten Orte und Spielerposition in die Serialisierung einbauen
- 

Arbeitsteilung (geplant):

Efe	Furkan
Planungsdokument Git repository Projektordner anlegen Schauplätze Lizenzen eintragen Projekttagbuch führen Dokumentation erweitern Stolpersteine & Lösungen dokumentieren Softwarevoraussetzungen und Versionen Architektur & Funktionsblöcke als Dokumentation als PDF und Projekt als .zip exportiert KI-Codes dokumentiert Zeitreiseort 1 programmieren Rätsel-Logik programmieren Rätsel programmieren Power Point erstellen Startpunkt mit Tagebuch programmieren	Sound & Grafik Projektordner anlegen Tagebuch-Logik und Zeitreiseprogramm Lizenzen eintragen Projekttagbuch führen Dokumentation erweitern Stolpersteine & Lösungen dokumentieren Skizzen Bedienungsanleitung Umsetzungsbeschreibung Probleme & Lösungen (Pflichtenheft) KI-Codes dokumentiert Power Point erstellen Zeitreise-Logik programmieren Bewegung programmieren Zeitreiseort 2 programmieren Story-Einschübe programmieren

Umsetzung (Pflichtenheft)

1. Welche Softwarevoraussetzungen werden benötigt?

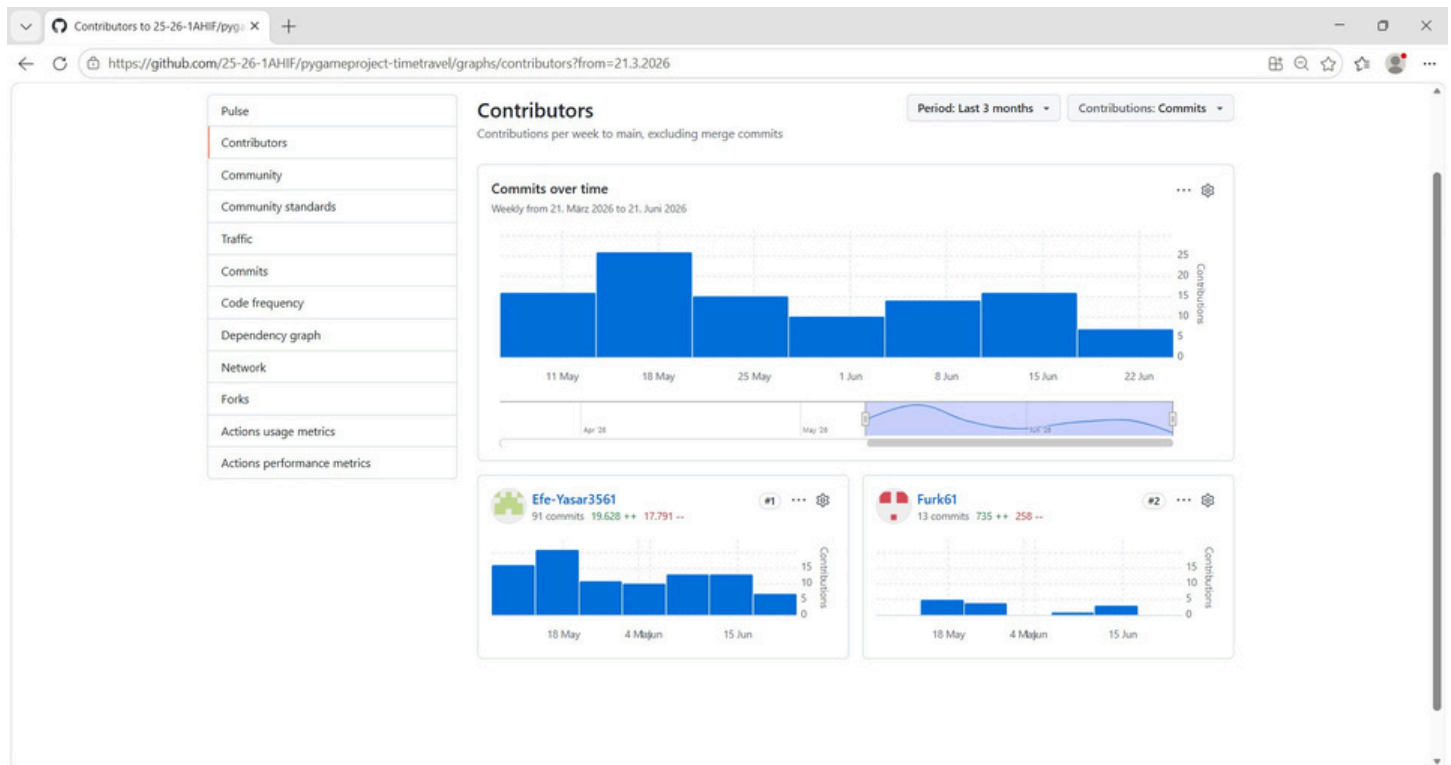
- pygame (alle Versionen)
- pyCharm (2025.2.2 oder Höher)
- siehe requirements.txt

2. Funktionsblöcke / Architektur:

- Die Screens sind fast alle eine einzige Funktion
- Nur Leaderboard ist eine Klasse
- Game_Variables.py hat eine classe GameObject
- mit dieser kann man ganz bequem Einzelne Objekte aus großen Tilesheets rausnehmen
- main.py dient als Screen-Router und handled die verschiedenen Screens
- main.py besteht nur aus 39 Zeilen aber der Rest sind alle um die 200 Zeilen

3. Beschreibung der Umsetzung:

Die Umsetzung verlief nicht so, wie geplant. Mithilfe von der GitHub Contributions sind folgende Zahlen zu Stande gekommen:



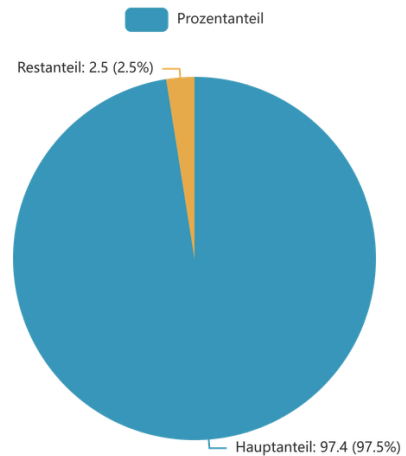
Das zeigt das Verhältnis von:

Efe Yasar: 91 commits, 19.628 ++ / 17.791 --

Furkan Yildiz: 13 commits, 735 ++ / 258 --

Falls diese Zahlen stimmen, entspricht das einer Arbeitsteilung von 97,4% (Efe Yasar) / 2,6% (Furkan Yildiz) der Codearbeit.

(Zahlen wurden mit KI berechnet. Datenschutz wurde gewahrt. KI hat keine graphischen Materialien, Namen oder sonst noch persönliche Daten des Gegenüber bekommen)



4. Probleme & Lösungen

Bugs sind im Laufe des Programmierens aufgetreten, wurden aber ohne dokumentiert zu werden aufgehoben.

Probleme:

- Das größte Problem welches im Spiel noch vorhanden ist, ist das Speichern mitten im Spiel, da falls man es speichert und dann wieder lädt, man die richtigen koordinaten hat, doch es eben immer im play_screen.py wieder spawnt. Die
- Falls man speichert und dann lädt, kann es sein, dass man ausserhalb des erlaubten Bewegungsraums im Attic sich wiederfinden könnte und dann müsste man das Spiel neustarten
- Die Buttons können gedrückt werden, aber man kann sich trotzdem bewegen währenddessen (optischer Feinschliff)

Lösungen:

- Um das Problem mit dem Speichern zu beheben, könnte man mit dem ScreenRouter zusammenarbeiten, sodass die Funktion immer weiß in welchem Screen der Player gespeichert hat und dann auch in dem Screen wieder Spawnen kann
- Problem zum Button könnte sein, die Zeit mitsamt der Bewegung einzuschränken, solange die Buttons/Anzeigen aktiv sind.

Projekttagebuch

Efe Yasar:

10.05.2026:

Main-File erstellt und ausgebaut mit def Main_Screen(...) und def Main().
Game_Variables erstellt

17.05.2026:

Buttons eingebaut
Hintergrundbild eingebaut
Beenden eingebaut

19.05.2026:

Play_Screen Funktion in main-Branch "Efe_main" erstellt

20.05.2026:

Tagebuch-Sprites erstellt:
Ein Bild
Ein umblättern Sprite
in main.py getestet
diary.py und sprites.py in assets/game erstellt

21.05.2026:

Diary vergrößert mithilfe von KI für die Nutzung beim Interagieren

22.05.2026:

main_screen ausgelagert auf main_screen.py
Tilemap - loader gebaut mithilfe von Hr. Bechtold
Tagebuch Sprite erstellt mit Ausklappen

29.05.2026:

Main Branch wurde aufgeräumt mit Hr. Bechtold
Logik aufgeräumt

30.05.2026:

Main von ca. 300 Zeilen Code auf 22 Zeilen aufgeräumt.
Play_Screen, Main_Screen & Pause_Screen wurden ausgelagert.
Den Code Aufgeräumt.

31.05.2026:

Background Sprites gelöscht
Grund: nicht Top Down und schwer aus dem Tilesheet rauszulesen
Nicht verwendet
1pxl Gap beim Tisch raus geschnitten
Unnötige Charaktere gelöscht,

player.py aus src.assets gelöscht
in src.game rein getan
player_variables auch da rein ergänzt um verwirrung vorzubeugen
Tisch grün gemacht damit das Tagebuch besser sichtbar ist
Tagebuch interaktion in den play_screen eingefügt
Speichern/Laden Bug gefixed

01.06.2026:

Play-Screen gebaut

Tilemap-leser angepasst

Tagebuchlogik so gemacht, dass wenn man 'E' drückt es öffnet und wenn man nochmal 'E' drückt dann schließt es sich.

alles etwas angepasst

07.06.2026:

Play_screen sehr ausgebaut

Map gezeichnet

Objekte erstellt

Haus platziert

Aufgeräumt

Bugs gefixed

Häuser gebaut

Brunnen versucht zu bauen

Brunnen gebaut, mit transparenz

interagieren so ausgebaut, dass es interaktiv ist

Wand und Obstacles usw. in Play_Screen gebaut, so dass sie funktionieren

In Medieval_Screen fehlt noch Häuser_kollisionen und Quiz

08.06.2026:

gefixed dass Player nicht aus der Map rausrennen kann

Map vergrößert,

verhindert dass Player in/durch Gebäude laufen kann

großen Bug mit transparenz gelöst

Brunnen und Castle automatisiert

14.06.2026:

Interact mit Häusern gemacht

Funktionen für Screens für diese Funktionen gebaut

15.06.2026:

Grundstruktur für Häuser gelegt

17.06.2026:

Rotes Haus final ausgebaut

Kerze als erstes Quest-Objekt platziert

Bilder/Avatare für die Quest-Objekte platziert

Player.move() so angepasst, dass er nicht über den Rand raus rennen kann

Vom roten Haus wieder zurück zum Medieval Screen gebaut

inventory.json erstellt, um das Quest-Inventar zu speichern

Quest Gegenstände speichern und beim Neustart des Spiels die Liste löschen

Kerze_bunt.png & Kerze_schwarz_weiß.png erstellt

Neues UI-Pack hinzugefügt altes entfernt

Projekttagebuch überarbeitet

19.06.2026:

Im Grey_House eine Chest platziert

in diese ein Quest-Objekt (Schild) eingefügt

wenn man es anklickt, hat man es eingesammelt

inventory.json logik umgeändert in Player Objekt

20.06.2026:

Player-Objekt für Inventar in GameVariables platziert

Schild_bunt.png und Schild_Schwarz_weiß erstellt

Kerze_schwarz_weiß Rahmen in grau umgeändert

main_screen, grey_house, medieval_screen und red_house angepasst

Portal_Text so umgeändert, dass falls der Player noch nicht alle eingesammelt hat, der Text:

"Collect Quest-Objects first Brudi" aufploppt

Wenn Player die Objekte hat, kann er ins Portal

Ein Welcome-Text in die Medieval Map eingebaut

Dafür GOT_ALL_ITEMS in GameVariables eingebaut damit es leichter ist

Ganze Castle Funktion gebaut

letztes Quest-Gegenstand eingebaut (Krone im Castle)

castle ausgeschmückt

3 Bilder erstellt

21.06.2026:

- Final fertig gebaut
- Bugs gefixed wie zum Beispiel Zeit zurücksetzen beim neuen Spiel
- Leaderboard gebaut
- Zeitmessung

Furkan Yildiz:

--

Bedienungsanleitung:

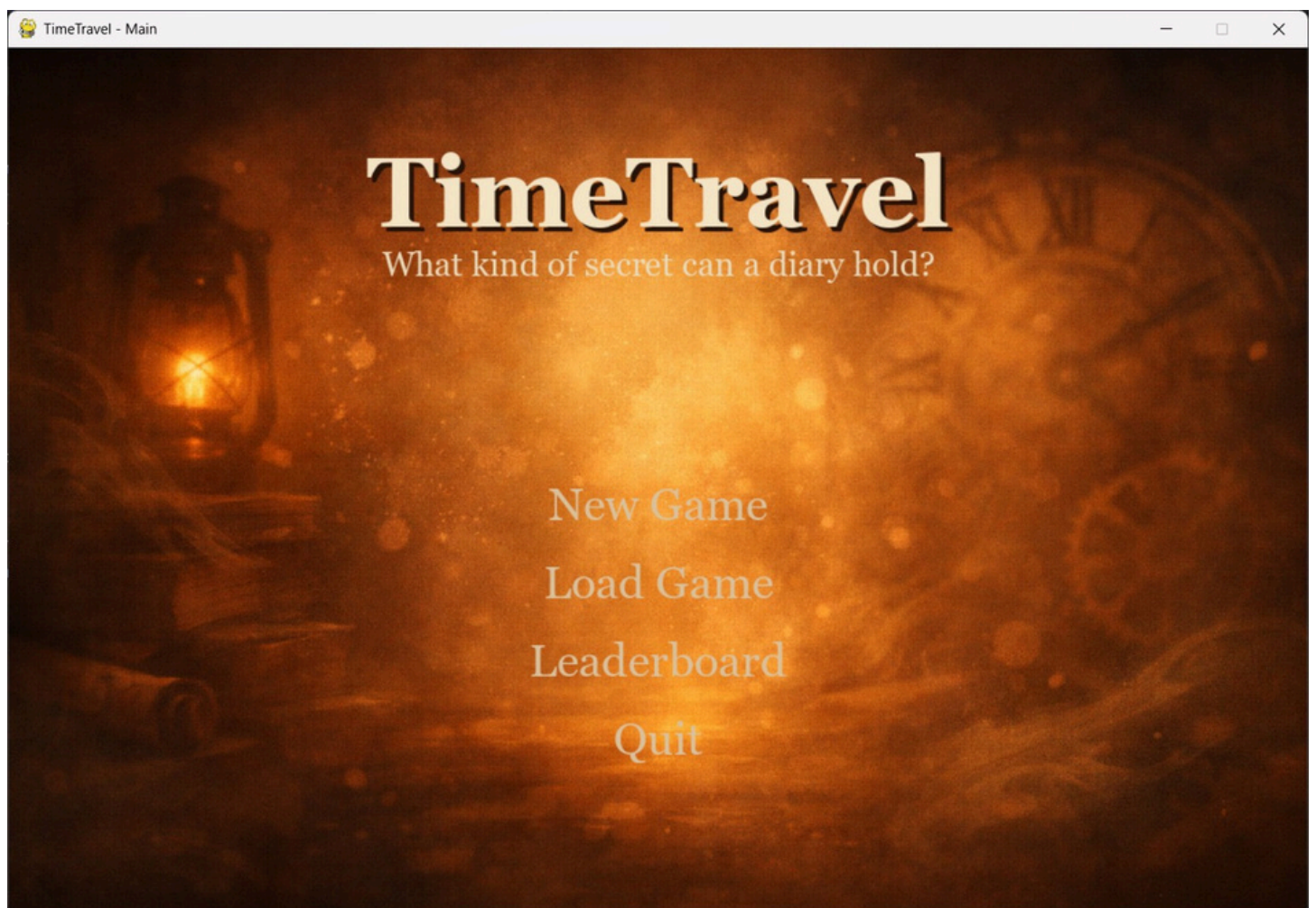
Spielstart

Nach dem Start des Spiels erscheint der **Main Screen**.

Hier kann der Spieler:

- ein neues Spiel starten
- ein Spiel laden
- das Leaderboard ansehen
- das Spiel beenden

Die Navigation erfolgt mit der **Maus**.



Ziel des Spiels

Der Spieler wird in einem Dachboden gespawnt und reist dann mithilfe eines Tagebuchs in eine mittelalterliche Umgebung und muss:

- **alle drei Gegenstände** (Krone, Schild, Kerze) finden
- sie in sein Inventar aufnehmen

- danach automatisch das Ziel erreichen

Die benötigte Zeit wird gemessen und im Leaderboard gespeichert.

Steuerung:

Bewegung

- **WASD** oder **Pfeiltasten** → Spieler bewegen

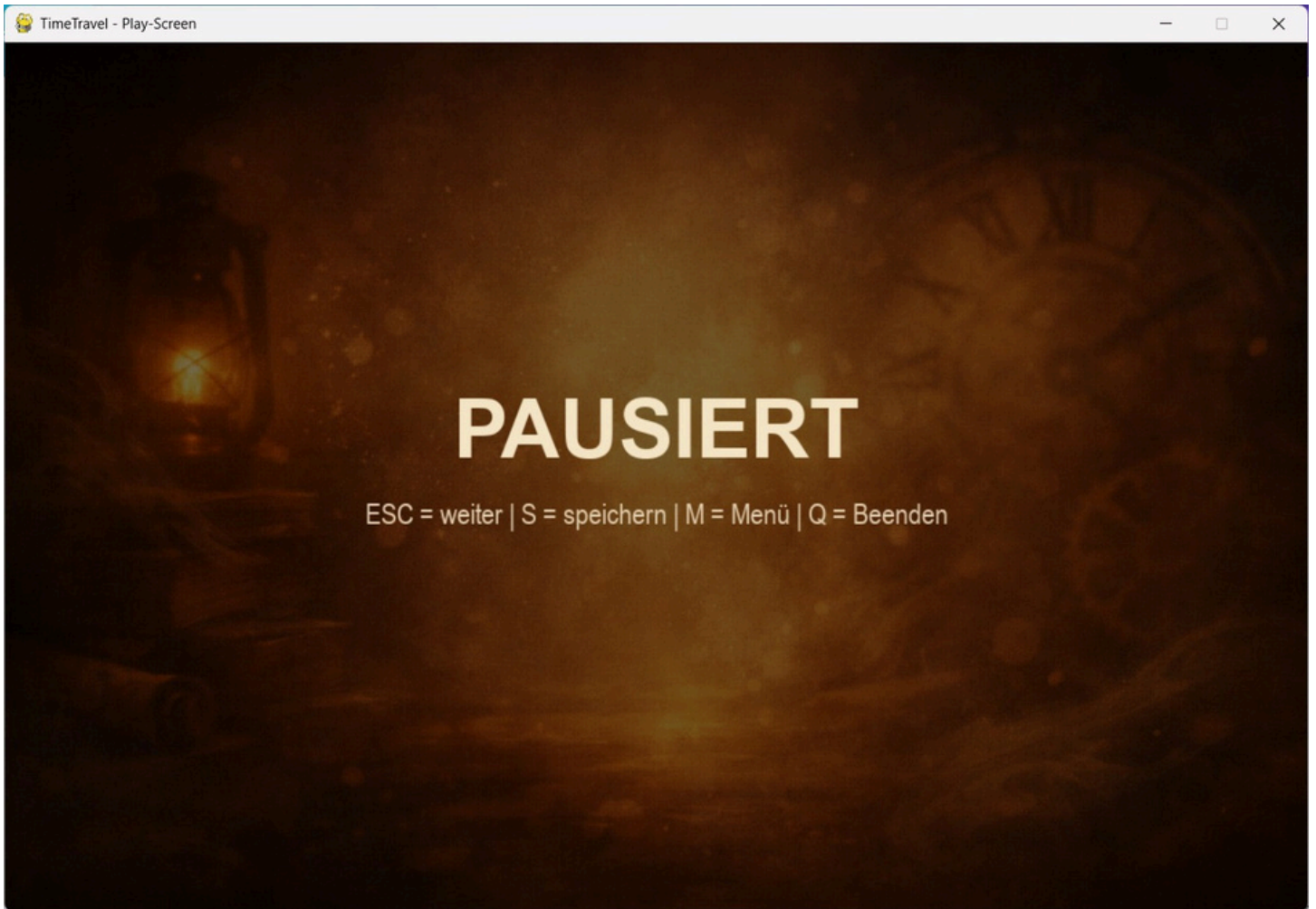
Interaktion

- **“E”-Taste** für die meisten Interaktionen (z.B. Tagebuch, Gebäude betreten/verlassen,...)
- **Linksklick** auf Schild → Gegenstand aufnehmen
- Buttons im UI → Aktionen ausführen (z. B. „Weiter“, „Start“, „Leaderboard“)



Menü

- **ESC** → **M** zurück zum Hauptmenü



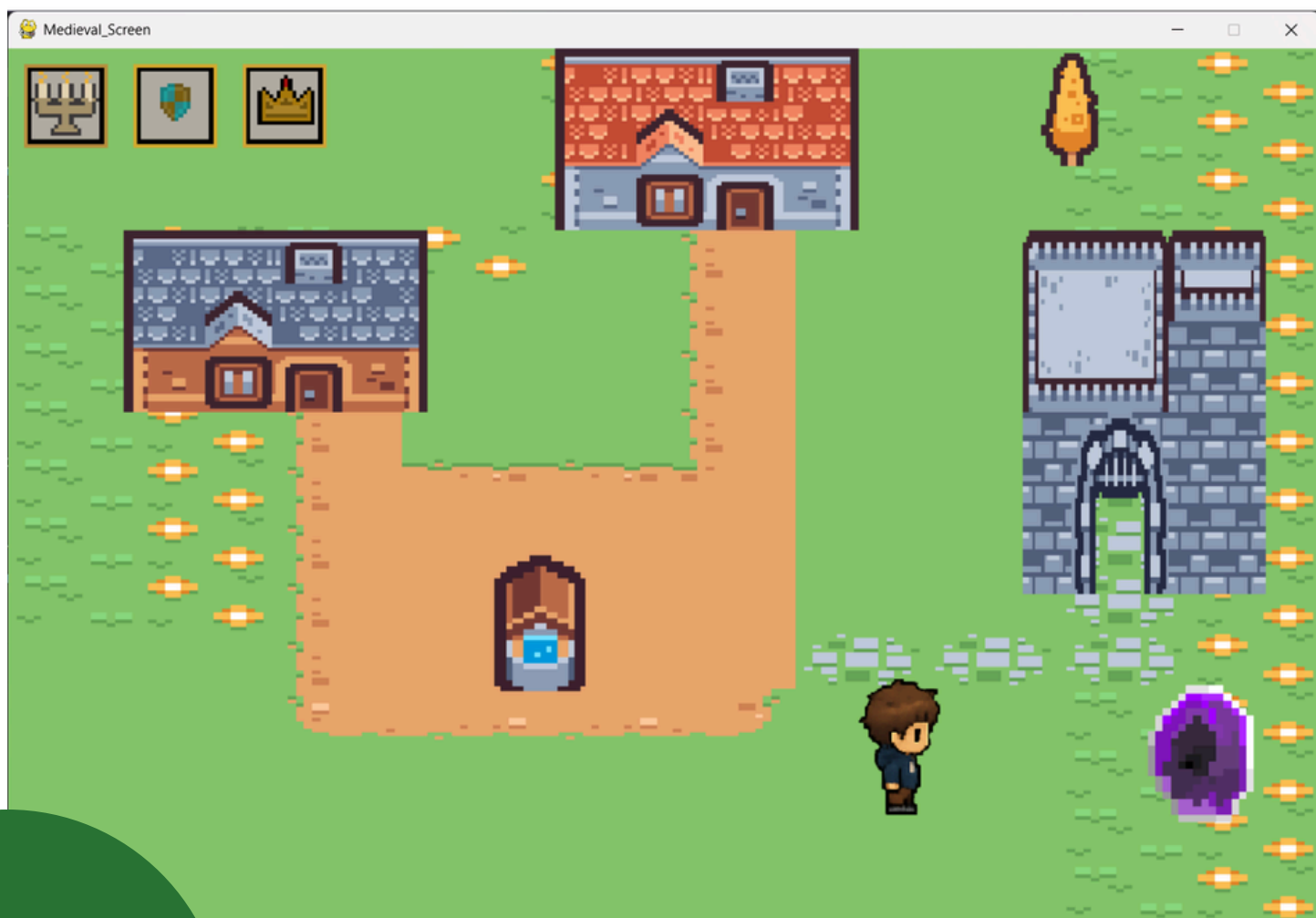
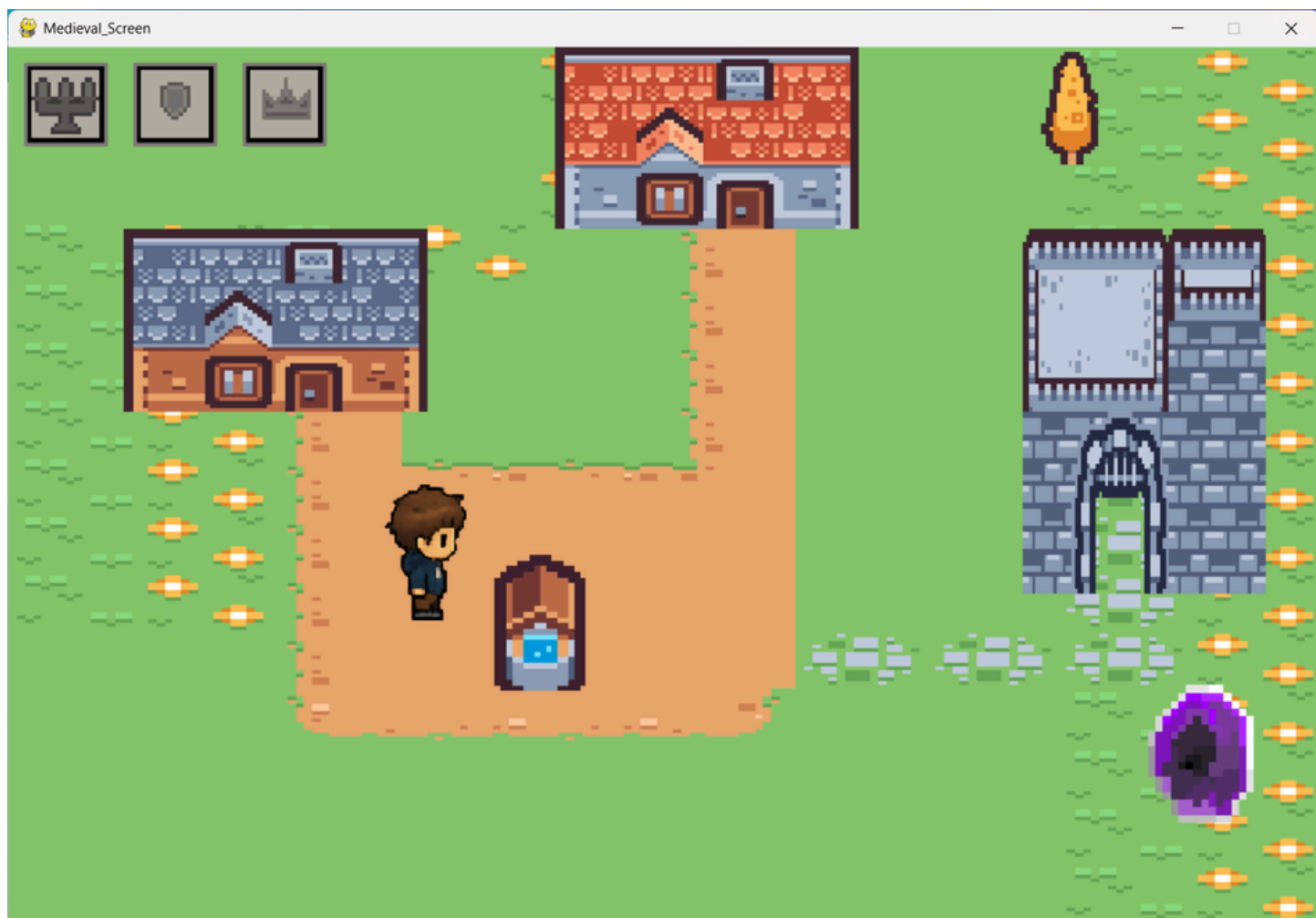
Spielablauf

1. Startscreen

- „New Game“ klicken, um das Spiel zu beginnen
- „Leaderboard“ klicken, um die Bestenliste anzusehen

2. Mittelalter-Level

- Der Timer startet automatisch beim ersten Betreten
- Der Spieler bewegt sich durch die Karte
- Klickt der Spieler auf einen Gegenstand, wird er ins Inventar gelegt
- Sobald alle drei Items gefunden wurden und der Player mit dem Portal interagiert, endet das Spiel automatisch



3. Namens eingabe

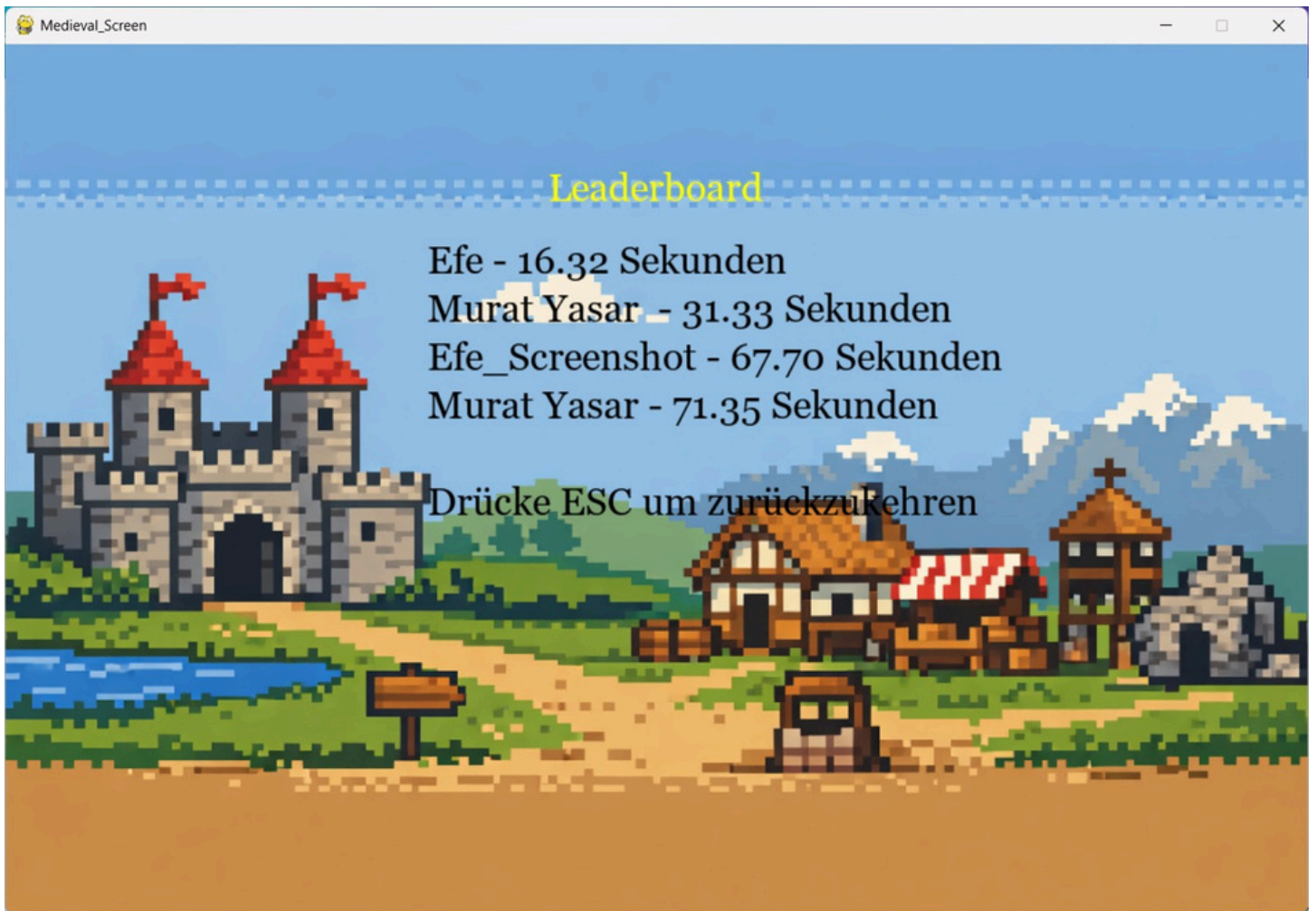
Nach dem Abschluss erscheint ein Eingabefeld:

- Spieler gibt seinen Namen ein
- Die Zeit wird im Leaderboard gespeichert



4. Leaderboard

- Zeigt die besten Zeiten
- Sortiert automatisch nach schnellster Zeit
- ESC → zurück zum Hauptmenü



Spielende

Das Spiel endet automatisch, sobald alle drei Gegenstände gefunden wurden.

Die benötigte Zeit wird gespeichert und kann im Leaderboard angesehen werden.

Quellen für verwendete Bilder und Graphiken

Lizenzen

Bilder

Main-Bild bei Startmenü:

Microsoft Copilot

<https://copilot.microsoft.com>

chest_closed, chest_open, crown_obj, Kerze_bunt, Kerze_Schwarz_Weiß, Krone_bunt, Krone_Schwarz_weiß, Schild_bunt, Schild_schwarz_Weiß, gesamte diary:

Efe Yasar mithilfe von Piskel

<https://www.piskelapp.com/p/create/sprite/?code=hero>

Audio

Sprites

Indoor - Sprite:

License (Creative Commons Zero, CC0)

<http://creativecommons.org/publicdomain/zero/1.0/>

Medieval Town:

License: (Creative Commons Zero, CC0)

<http://creativecommons.org/publicdomain/zero/1.0/>

UI_Pack:

License: (Creative Commons Zero, CC0)

<http://creativecommons.org/publicdomain/zero/1.0/>

Player:

ChatGPT

<https://chatgpt.com>